```
    --from-literal=spring.datasource.username=<postgres_username> \
    --from-literal=spring.datasource.password=<postgres_password>
```

Similarly, create a Secret for Order Service. Pay attention to the slightly different syntax required by Spring Data R2DBC for the URL:

```
$ kubectl create secret generic polar-postgres-order-credentials \
    --from-literal="spring.flyway.url=
➥jdbc:postgresql://<postgres_host>:<postgres_port>/polardb_order" \
    --from-literal="spring.r2dbc.url=
➥r2dbc:postgresql://<postgres_host>:<postgres_port>/polardb_order?
➥ssl=true&sslMode=require" \
    --from-literal=spring.r2dbc.username=<postgres_username> \
    --from-literal=spring.r2dbc.password=<postgres_password>
```

That's it for PostgreSQL. In the next section, you'll see how to initialize Redis using DigitalOcean.

B.3 Running Redis on DigitalOcean

In most of the book, you've been running Redis instances as containers, both in Docker and in your local Kubernetes cluster. In production we'd like to take advantage of the platform and use a managed Redis service provided by DigitalOcean (https://docs.digitalocean.com/products/databases/redis/).

Once again, since we followed the 15-Factor methodology, we can swap the Redis backing service used by Edge Service without changing anything in the application code. We'll only need to change a few configuration properties for Spring Boot.

First, create a new Redis server named `polar-redis` as shown in the following code snippet. We'll use Redis 7, which is the same version we used for development and testing. Remember to replace `<your_region>` with the geographical region you'd like to use. It